

基于 Huffman 编码的文件压缩与解压缩实验报告

1. 问题背景描述

随着数字化信息的爆炸式增长，数据压缩技术在存储和传输中扮演着重要角色。Huffman 编码是一种经典的无损数据压缩算法，由 David A. Huffman 于 1952 年提出。该算法根据字符出现的频率构建最优前缀码，使得高频字符使用短编码，低频字符使用长编码，从而达到整体压缩的目的。

本实验旨在利用 Huffman 编码实现一个简单的文本文件压缩与解压缩工具，支持英文和中文文本。同时，通过实验发现仅使用 Huffman 编码对中文文本压缩存在的不足，并调研主流压缩软件的改进思路，深入理解实际工业级压缩算法的设计思想。

code:<https://github.com/dadaozhichen/huffman>

2. 算法代码设计

2.1 总体结构

程序包含以下核心模块：

- **Node 类**：Huffman 树的节点，存储字符、频率及左右孩子指针。
- **Huffman 类**：封装了编码表、根节点及 `encode / decode` 方法。
- **构建函数** `build(freq)`：根据字符频率列表构建 Huffman 树并生成编码表。
- **频率统计** `count_freq(s)`：统计输入字符串中每个字符的出现次数。
- **比特流打包/解包** `pack / unpack`：将比特字符串转换为字节流，并处理填充位。
- **压缩/解压缩主函数** `compress / decompress`：完成文件读写、频率表存储及编解码流程。
- **命令行接口**：通过 `argparse` 提供 `--encode/-e` 和 `--decode/-d` 两种模式，以及 `--input/-i` 和 `--output/-o` 参数。

2.2 关键实现细节

2.2.1 Huffman 树的声明

Node 类

作用：表示 Huffman 树的节点。

实现思路：

- `__init__(self, str, freq, l=None, r=None)`：
保存字符（叶节点为实际字符，内部节点为 `None`）、频率、左孩子、右孩子。
- `__lt__(self, other)`：
重载 `<` 运算符，使节点可按频率比较大小，以便放入 `heapq` 最小堆中。

设计意图：

内部节点无字符值，仅用于合并两个子节点；可比较性支持贪心合并时每次取出两个最小频率节点。

Huffman 类

作用：封装 Huffman 树和编码表，提供编码/解码接口。

实现思路：

- `__init__(self, root, code: dict[str, str]):`
存储根节点和编码字典（字符 → 二进制串）。
- `encode(self, s: str):`
遍历输入字符串，对每个字符查表获取编码，用 `''.join` 连接成完整的比特串。
- `decode(self, s: str):`
遍历比特串，从根节点开始：遇 `'0'` 走向左孩子，遇 `'1'` 走向右孩子。
到达叶节点时输出字符，并将当前指针重置为根节点，继续处理剩余比特。

设计意图：

编码采用查表法实现 $O(n)$ 复杂度；解码利用 Huffman 树的前缀特性，无需分隔符。

```
class Node:
    def __init__(self, str, freq, l=None, r=None) -> None:
        self.freq = freq
        self.char = str
        self.l = l
        self.r = r
    def __lt__(self, other):
        return self.freq < other.freq

class Huffman:
    def __init__(self, root, code: dict[str, str]) -> None:
        self.root = root
        self.code = code
    def encode(self, str):
        ans = ''.join([self.code[x] for x in str])
        return ans
    def decode(self, str):
        ans = ''
        p = self.root
        for v in str:
            if v == '0':
                p = p.l
            else:
                p = p.r
            if p.char != None:
                ans = ans + p.char
                p = self.root
        return ans
```

2.2.2 Huffman 树构建（使用最小堆）

`build(freq: list[tuple[str, int]]) -> Huffman`

作用：根据字符频率列表构建 Huffman 树，生成编码表，返回 `Huffman` 对象。

实现思路：

1. 将频率列表转换为 `(freq, Node(char, freq))` 的列表，并堆化（最小堆）。
2. 当堆大小 > 1 时循环：
 - 弹出两个最小节点 `(f1, n1)` 和 `(f2, n2)`。
 - 创建新节点 `Node(None, f1+f2, n1, n2)`。
 - 将新节点 `(f1+f2, new_node)` 压回堆中。
3. 弹出最后一个节点作为根节点。
4. 深度优先遍历 Huffman 树，左分支追加 `'0'`，右分支追加 `'1'`，遇到叶节点时记录编码到字典中。
5. 返回 `Huffman(root, code)`。

设计意图：

最小堆保证每次合并频率最小的两个节点，符合 Huffman 贪心策略；DFS 递归生成前缀码。

```
def build(freq: list[tuple[str, int]]) -> Huffman:
    heap = [(freq, Node(char, freq)) for char, freq in freq]
    heapify(heap)
    while len(heap) > 1:
        f1, n1 = heappop(heap)
        f2, n2 = heappop(heap)
        new_node = Node(None, f1 + f2, n1, n2)
        heappush(heap, (f1 + f2, new_node))
    _, root = heappop(heap)

    code = {}
    def dfs(u, cur_code):
        if u.char is not None:
            code[u.char] = cur_code
        else:
            dfs(u.l, cur_code + '0')
            dfs(u.r, cur_code + '1')
    dfs(root, '')
    return Huffman(root, code)
```

2.2.3 统计出现频率

`count_freq(s: str) -> list[tuple[str, int]]`

作用：统计输入字符串中每个字符的出现频率。

实现思路：

- 初始化空字典 `m`。
- 遍历字符串每个字符 `c`： `m[c] = m.get(c, 0) + 1`。
- 返回 `list(m.items())`，即 `[(char, freq), ...]` 列表。

设计意图：

使用字典手动统计，减少外部依赖；返回列表便于后续堆构建。

```
def count_freq(s:str):
    m = dict()
    for c in s:
        v = m.get(c,0)
        m[c] = v+1
    return [x for x in m.items()]
```

2.2.4 打包与解包

pack(bit_str: str) -> bytes

作用：将 Huffman 编码后的比特字符串打包为字节流，并记录填充信息。

实现思路：

1. 记录原始比特串长度 `original_len`。
2. 计算所需填充位数 `padding = (8 - original_len % 8) % 8`。
3. 若填充位数不为 0，则在比特串末尾补 `'0' * padding`。
4. 按每 8 位一组转换成整数，添加到 `bytearray` 中。
5. 构造头部：`padding`（1 字节）+ `original_len`（4 字节，大端序）。
6. 返回头部 + 数据体。

设计意图：

字节对齐便于文件存储；头部记录填充位数和原始长度，确保解包时能还原精确的比特串。

unpack(data: bytes) -> str

作用：从打包的字节流中恢复原始的比特字符串。

实现思路：

1. 读取第一个字节作为 `padding`。
2. 读取第 2~5 字节，转换为整数 `original_len`。
3. 遍历第 6 字节到末尾，将每个字节格式化为 8 位二进制字符串，拼接成完整的比特串。
4. 截取前 `original_len` 位返回。

设计意图：

与 `pack` 对称，利用头部信息精确还原填充前的原始比特串。

```
def pack(bit_str: str) -> bytes:
    original_len = len(bit_str)
    padding = (8 - original_len % 8) % 8
    if padding:
        bit_str += '0' * padding
    data = bytearray()
    for i in range(0, len(bit_str), 8):
        byte = int(bit_str[i:i+8], 2)
        data.append(byte)
    header = bytes([padding]) + original_len.to_bytes(4, 'big')
```

```

return header + bytes(data)

def unpack(data: bytes) -> str:
    padding = data[0]
    original_len = int.from_bytes(data[1:5], 'big')
    full_bit_str = ''
    for b in data[5:]:
        full_bit_str += f'{b:08b}'
    return full_bit_str[:original_len]

```

2.2.5 压缩与解压缩

compress(src_path: str, dst_path: str)

作用：压缩文件。

实现思路：

1. 以只读文本模式打开源文件（UTF-8 编码），读取全部内容。
2. 调用 `count_freq` 统计字符频率。
3. 调用 `build` 构建 Huffman 树并获取编码器。
4. 调用 `tree.encode` 将文本编码为比特字符串。
5. 调用 `pack` 将比特字符串打包为字节流。
6. 以二进制写模式打开目标文件：
 - 写入频率表长度（2 字节）。
 - 遍历频率表，先写入字符的 Unicode 码点（4 字节），再写入频率值（4 字节）。
 - 写入打包后的压缩数据。

设计意图：

将频率表与压缩数据一同存储，解压时可重建相同的 Huffman 树；文件格式自包含。

decompress(in_path: str, out_path: str)

作用：解压缩文件。

实现思路：

1. 以二进制读模式打开压缩文件。
2. 读取前 2 字节，得到频率表项数 `freq_len`。
3. 循环 `freq_len` 次，每次读取 4 字节作为 Unicode 码点，再读取 4 字节作为频率值，将 `(chr(code), freq)` 存入列表。
4. 调用 `build(freq)` 重建 Huffman 树及编码器。
5. 读取剩余全部数据作为压缩数据体。
6. 调用 `unpack` 将字节流还原为比特字符串。
7. 调用 `tree.decode` 将比特字符串还原为原始文本。

8. 以写文本模式（UTF-8 编码）将原始文本写入输出文件。

设计意图：

完全对称于压缩过程，确保无损还原；重建频率表即重建 Huffman 树，无需额外传输树结构。

```
def compress(src_path: str, dst_path: str):
    with open(src_path, 'r', encoding='utf-8') as f:
        text = f.read()
        freq = count_freq(text)
        tree = build(freq)
        bit_str = tree.encode(text)
        packed_data = pack(bit_str)
    with open(dst_path, 'wb') as f:

        f.write(len(freq).to_bytes(2, 'big'))
        for ch, fr in freq:

            f.write(ord(ch).to_bytes(4, 'big'))

            f.write(fr.to_bytes(4, 'big'))
        f.write(packed_data)

def decompress(in_path: str, out_path: str):
    with open(in_path, 'rb') as f:
        freq_len = int.from_bytes(f.read(2), 'big')
        freq = []
        for _ in range(freq_len):
            code = int.from_bytes(f.read(4), 'big')
            ch = chr(code)
            fr = int.from_bytes(f.read(4), 'big')
            freq.append((ch, fr))

        tree = build(freq)
        compressed_data = f.read()

        bit_str = unpack(compressed_data)
        original_text = tree.decode(bit_str)

    with open(out_path, 'w', encoding='utf-8') as f:
        f.write(original_text)
```

2.2.6 命令行接口

```
def main():
    parser = argparse.ArgumentParser('encode or decode')
    group = parser.add_mutually_exclusive_group(required=True)
    group.add_argument('--encode', '-e', action='store_true')
    group.add_argument('--decode', '-d', action='store_true')
    parser.add_argument('--input', '-i', type=str, required=True)
    parser.add_argument('--output', '-o', type=str, default=None)
    args = parser.parse_args()
```

```
if args.encode:
    output_path = args.output if args.output else args.input[:-4]+'.huff'
    compress(args.input,output_path)
    print(f"The compressed file is in {output_path}")
else:
    if args.output:
        output_path = args.output
    else:
        output_path = args.input[:-5]+' .txt'
    decompress(args.input,output_path)
    print(f"The decompressed file is in {output_path}")
```

3.运行结果

3.1 测试环境

- 操作系统：Ubuntu 26.04 LTS
- Python 版本：3.13
- 测试文件：
 - in2.txt：纯中文文本
 - in3.txt：纯英文文本两个均为相同意思的文本

3.2 压缩效果对比

文件	原始大小	压缩后大小	压缩率	体积变化
in2.txt	1,270 B	2,296 B	180.8%	膨胀 80.8%
in3.txt	1,712 B	1,474 B	86.1%	节省 13.9%

结果分析：

- 英文文件：压缩后体积减小（约 14% 节省）。虽然节省比例不高，但验证了 Huffman 编码的有效性。空间节省有限是因为测试文件较小，频率表头部开销占比较大。
- 中文文件：压缩后体积反而膨胀至原来的 1.8 倍。原因如下：
 1. 头部开销远超收益：文件仅 1270 字节，但其中包含 200+ 个不同的汉字字符。每个字符在头部占用 8 字节（4 字节码点 + 4 字节频率），总头部约 $2 + 200 \times 8 = 1602$ 字节，已经大于原始文件。而压缩后的数据体因编码长度均匀（接近固定长度），几乎不节省空间，最终总体积反而增大。
 2. 频率分布平滑：中文字符频率差异小，Huffman 树平衡，平均码长接近 $\log_2(200) \approx 8$ 比特，与原始 UTF-8 编码（通常 3 字节/汉字）相比，编码后的比特数反而可能更多。

3.3 解压缩验证

对 `temp.huff` 解压得到 `out2.txt` 和 `out3.txt`，与原始文件完全一致（通过 `fc /b` 比对），证明算法正确性，压缩膨胀不影响可逆性。

3.4 重要结论

纯 Huffman 编码不适用于小体积中文文本，甚至会产生负压缩。对于实用压缩工具，必须结合字典压缩（LZ77）并针对中文采用字节级处理（如 UTF-8 字节流）或预置词典。

4. 问题与总结

4.1 核心问题：Huffman 编码对中文文本压缩失效

实测数据显示，使用纯 Huffman 编码压缩一个 1270 字节的中文文本，压缩后体积反而膨胀到 2296 字节（膨胀率 80.8%）；而压缩 1712 字节的英文文本，仅节省 13.9% 空间。这说明：

- 对于英文：Huffman 编码仍有一定压缩效果，但因测试文件较小，频率表头部开销占比较高，收益有限。
- 对于中文：出现严重的负压缩（输出比输入还大），导致算法完全不可用。

4.2 失败原因分析

（1）频率表头部开销过大

本实验的文件格式约定：每个不同字符需在头部存储 4 字节 Unicode 码点 + 4 字节频率值，再加上 2 字节的表项数。对于包含 200+ 个不同汉字的中文文本，头部占用约 $2 + 200 \times 8 = 1602$ 字节，已超过原始文件大小（1270 字节）。而数据体部分几乎没有压缩收益，导致整体膨胀。

（2）中文字符集大且频率均匀

英文仅 26 个字母，频率差异悬殊（如 `e` 远高于 `z`），Huffman 树极不平衡，高频字符可获极短编码（如 2~3 比特）。而中文常用汉字超过 3000 个，即使一篇短文也可能出现上百个不同字符，每个字符出现次数相近，Huffman 树趋近平衡，平均码长接近 $\log_2(N) \approx 8$ 比特。但原始 UTF-8 编码下，一个汉字通常占 3 字节（24 比特），8 比特的 Huffman 码看似更短，然而加上头部开销后反而得不偿失。

（3）缺乏上下文建模

Huffman 编码是零阶熵编码，仅统计单字符概率，无法利用中文词语内部的强相关性（如“葡萄”中的两个字符几乎总是相伴出现）。这类冗余需通过字典压缩（LZ77/LZ78）来消除。

4.3 主流压缩软件的解决方案

工业级压缩工具（ZIP、7-Zip、gzip、RAR）从不单独使用 Huffman 编码，而是采用混合模型。以 DEFLATE 算法（ZIP/gzip 核心）为例：

阶段	方法	对中文的作用
1	LZ77 滑动窗口	查找重复字节序列（如词语、短语、标点模式），替换为（距离，长度）引用。可大幅压缩中文文本中反复出现的词语。
2	Huffman 二次编码	对 LZ77 输出的字面量、匹配长度、距离进行动态 Huffman 编码，进一步缩减冗余。
3	预置词典（可选）	7-Zip 的 LZMA 支持自定义词典；Brotli 内置常用中文短语，小文件也能获得良好压缩比。

此外，现代压缩算法还采用：

- **算术编码 / 区间编码**：编码效率比 Huffman 更接近熵极限（如 LZMA）。
- **字节级处理**：以 UTF-8 字节流为单位，而非 Unicode 字符，可同时兼容 ASCII 和多字节字符，并利用字节级重复模式。

4.4 本实验的改进方向

基于上述分析，可为当前程序加入以下改进：

1. **增加 LZ77 预处理**：对输入文本先进行重复字符串查找与引用替换，再将中间结果送入 Huffman 编码。
2. **改用自适应 Huffman 编码**：不预先存储频率表，而在一遍扫描中动态更新树结构，彻底消除头部开销。
3. **按字节处理而非字符**：读取文件为二进制数据（UTF-8 字节流），使算法天然支持任何语言，并能利用字节值 0~255 的频率差异（通常重复字节模式更多）。
4. **小文件特殊处理**：当文件小于阈值（如 4KB）且压缩后体积不减反增时，直接存储原始数据并标记为“未压缩”。

4.5 总结与反思

通过本次实验，我们不仅掌握了 Huffman 编码的实现细节，更深刻理解了“**算法脱离数据特征便无用武之地**”的道理。一个在英文上有效的熵编码，面对中文时却产生负面效果，这警示我们：

- 设计压缩系统前，必须先分析目标数据的统计特性（字符集大小、频率分布、相关性）。
- 实际应用中的压缩方案都是多种技术的组合（字典 + 熵编码 + 自适应模型），单一算法几乎无法通用。
- 评估压缩算法时，必须考虑**头部开销**和**小文件场景**，否则可能得出误导性结论。

未来改进中，可尝试将当前程序扩展为 LZ77 + Huffman 的简化版 DEFLATE，并测试中文文本的压缩效果是否得到根本改善。

5.附录

测试文本 `in2.txt`：

早上七点二十，闹钟响第三遍才摸手机。很多消息没空看，赶着开早会。匆忙洗漱出门。
电梯里大爷说上班辛苦，我点头。骑车去公司，风很凉。
同事小李递给我一杯美式，记得我爱喝这个。心里一暖。到工位，处理了几封紧急邮件，泡茶发呆了一会儿。想起大学时晒太阳聊天的日子。
九点半开早会，我汇报方案，有人问季节性波动，我答了。老板说方案不错但要打磨，意思是要改。
中午改文档忘了吃饭，小李叫我。去食堂只剩西红柿炒蛋和紫菜汤，随便吃几口继续干活。
下午两点客户电话会，沟通一小时，需求对齐。眼睛酸，去茶水间看到绿萝长新叶。
三点多快递让取包裹，可能是买的书。四点在同事群转发段子给大学室友群，没人回。
五点半下班没人走，我忙到六点二十。天黑风冷，买了杯热奶茶，三分糖加燕麦。
等奶茶时刷到大学同学婚礼照，点赞祝福。回家路上看到小孩玩滑梯。
到家我妈打电话问吃饭没，我说没。她让我煮面别点外卖。我用鸡蛋青菜下了碗面。
八点看书，九点半洗漱，十点多困了，关灯想着明天的事，慢慢睡着。

测试文本 `in3.txt`：

At 7:20, my alarm rang for the third time. I grabbed my phone—unread messages from work, the courier, and my mom. No time to listen; an important morning meeting couldn't wait. I rushed through washing up, grabbed a bread, and left.
In the elevator, a neighbor said, "Off to work? Tough, huh?" I nodded. Outside, I shared a bike and listened to music. The cool breeze woke me up.
At the office, my colleague handed me a coffee—American, no sugar, just the way I like it. A small warmth. I went to my desk, replied to urgent emails, and stared out the window for a moment.
At 9:30, the meeting started. I presented my proposal, answered a question about seasonal fluctuations, and got a nod from the boss. Afterward, he said it was good but needed polishing—code for "revise it."
Back at my desk, I revised. Lunchtime came and went. I ended up with tomatoes and eggs at the canteen.
At 2 PM, a client call dragged on for an hour. Finally, we aligned. I sighed, rubbed my eyes, and saw new leaves on the office plant.
At 3, the courier left a package—probably the book I ordered. At 4, a funny meme in the work group. I forwarded it to my college buddies; no reply.
At 5:30, no one left. I kept working until 6:20. Headed out into the dark, cold evening. Bought a warm milk tea—30% sugar, with oats.
Scrolling through Moments, I saw a college friend's wedding photos. I liked the post. Walking home, I passed kids on a slide.
At home, my mom called. "Have you eaten?" "Not yet." "Cook some noodles, don't always order takeout." I made noodles with eggs and greens.
At 8 PM, I read a few pages of my book. At 9:30, I brushed my teeth and scrolled my phone. By 10, I turned off the light and fell asleep.

测试脚本 `test.sh`：

```
INPUT="in3.txt"#可改为in2.txt
COMPRESSED="temp.huff"
OUTPUT="out3.txt"#可改为in2.txt

echo "压缩 $INPUT -> $COMPRESSED"
```

```
python huffman.py -e -i "$INPUT" -o "$COMPRESSED"

echo "解压 $COMPRESSED -> $OUTPUT"
python huffman.py -d -i "$COMPRESSED" -o "$OUTPUT"

echo "====="
echo "原文件大小:      $(stat -c %s "$INPUT") 字节"
echo "压缩后大小:      $(stat -c %s "$COMPRESSED") 字节"
echo "解压后大小:      $(stat -c %s "$OUTPUT") 字节"
echo "====="

if cmp -s "$INPUT" "$OUTPUT"; then
    echo "解压后的文件与原文件完全一致"
else
    echo "解压后的文件与原文件不同"
fi
```